DEPARTMENT OF ELECTRONIC & TELECOMMUNICATION ENGINEERING

UNIVERSITY OF MORATUWA

EN3160: Image Processing and Machine Vision

Assignment 2: Fitting and Alignment

Name: Aroshana H. A. P.
Index Number: 230058N
Github Repository: 230058n_a02

October 18, 2026

---

1 Blob Detection using Laplacian of Gaussians

1.1 Procedure

Blob detection was performed on the provided sunflower image using the Laplacian of Gaussian (LoG) operator combined with scale space extrema detection.

1.2 Implementation and Results

[1]:
```
import cv2 as cv
import numpy as np
import matplotlib.pyplot as plt
from scipy.ndimage import gaussian_laplace, maximum_filter

# load the image
img_path = 'images/the_berry_farms_sunflower_field.jpeg'
img_color = cv.imread(img_path, cv.IMREAD_REDUCED_COLOR_4)

if img_color is not None:
    img_gray = cv.cvtColor(img_color, cv.COLOR_BGR2GRAY)
    img_gray = img_gray / 255.0

    sigma_min = 1.0       # scale space parameters
    sigma_max = 12.0
    k = 1.1
    sigmas = []
    scale_space = []

    # construct the scale space using normalized LoG
    s = sigma_min
    while s <= sigma_max:
        sigmas.append(s)
        log_img = gaussian_laplace(img_gray, sigma=s) * (s**2)
        scale_space.append(np.abs(log_img))
```

```
        s *= k
    scale_space = np.stack(scale_space, axis=-1)

    # extrema detection
    local_max = maximum_filter(scale_space, size=5)

    # apply a threshold to discard weak responses
    threshold = 0.31
    extrema_mask = (scale_space == local_max) & (scale_space > threshold)

    # extract blob coordinates and radii
    y_coords, x_coords, scale_idxs = np.where(extrema_mask)
    img_out = img_color.copy()
    blobs = []

    for y, x, s_idx in zip(y_coords, x_coords, scale_idxs):
        sigma = sigmas[s_idx]
        radius = int(np.sqrt(2) * sigma)
        blobs.append((x, y, radius, scale_space[y, x, s_idx]))
        cv.circle(img_out, (x, y), radius, (0, 255, 0), 2)

    # output the range of sigma and the parameters of the largest circles
    blobs.sort(key=lambda b: b[2], reverse=True)
    print(f"Range of sigma values used: {sigma_min} to {sigmas[-1]:.2f}")
    print("Parameters of the largest circles (x, y, radius):")
    for b in blobs[:5]:
        print(f"Center: ({b[0]}, {b[1]}), Radius: {b[2]}")

    # display the result
    fig, axes = plt.subplots(1, 2, figsize=(8, 4))

    axes[0].imshow(cv.cvtColor(img_color, cv.COLOR_BGR2RGB))
    axes[0].set_title('Original Image')
    axes[0].axis('off')

    axes[1].imshow(cv.cvtColor(img_out, cv.COLOR_BGR2RGB))
    axes[1].set_title('Detected Circles (LoG Blob Detection)')
    axes[1].axis('off')

    plt.tight_layout()
    plt.show()
else:
    print("Error: Image not found.")
```

```
Range of sigma values used: 1.0 to 11.92
Parameters of the largest circles (x, y, radius):
Center: (0, 275), Radius: 10
Center: (282, 338), Radius: 10
```

```
Center: (106, 255), Radius: 8
Center: (179, 327), Radius: 8
Center: (178, 261), Radius: 7
```



1.3 Discussion

By evaluating the scale-space extrema, distinct circular structures corresponding to the sunflowers were successfully isolated. A specific range of σ (sigma) values was carefully selected to match the varying spatial dimensions of the flowers in the field. Based on this extrema analysis, the center coordinates and radii of the largest detected circles were accurately extracted.

2 RANSAC Line and Circle Fitting

2.1 Procedure

A set of noisy 2D points was generated, containing both linear and circular structures. The Random Sample Consensus (RANSAC) algorithm was manually implemented to sequentially extract these models. First, the best fit line was estimated, and its consensus inliers were removed from the dataset. Subsequently, the remaining points were used to estimate the best fit circle.

2.2 Implementation and Results

[ ]:
```python
import numpy as np
from scipy.optimize import minimize
import matplotlib.pyplot as plt

# data generation
np.random.seed(0)
```

```
N = 100
half_n = N // 2
r = 10
x0_gt, y0_gt = 2, 3
s_noise = r / 16
t = np.random.uniform(0, 2*np.pi, half_n)
n = s_noise * np.random.randn(half_n)
x, y = x0_gt + (r + n)*np.cos(t), y0_gt + (r + n)*np.sin(t)
X_circ = np.hstack((x.reshape(half_n, 1), y.reshape(half_n, 1)))

s_line = 1.
m, b_int = -1, 2
x = np.linspace(-12, 12, half_n)
y = m*x + b_int + s_line*np.random.randn(half_n)
X_line = np.hstack((x.reshape(half_n, 1), y.reshape(half_n, 1)))

X = np.vstack((X_circ, X_line))

# RANSAC line fitting
def fit_line_tls(points):
    """total least squares line fitting using SVD"""
    mean = np.mean(points, axis=0)
    centered = points - mean
    U, S, Vt = np.linalg.svd(centered)
    a, b = Vt[-1, :]
    d = a * mean[0] + b * mean[1]
    if d < 0:
        a, b, d = -a, -b, -d
    return a, b, d

def ransac_line(points, iterations=1000, threshold=1.5, consensus_min=20):
    best_inliers, best_model, best_sample = [], None, None
    for _ in range(iterations):
        sample = points[np.random.choice(len(points), 2, replace=False)]
        a, b, d = fit_line_tls(sample)

        # Calculate normal distance
        distances = np.abs(points[:, 0]*a + points[:, 1]*b - d)
        inliers_idx = np.where(distances < threshold)[0]

        if len(inliers_idx) > len(best_inliers):
            best_inliers, best_model, best_sample = inliers_idx, (a, b, d),␣
↪sample

    if len(best_inliers) >= consensus_min:
        best_model = fit_line_tls(points[best_inliers]) # Refit
    return best_model, best_inliers, best_sample
```

```
line_model, line_inliers_idx, line_sample = ransac_line(X)

# RANSAC circle fitting
# Isolate remnants
mask = np.ones(len(X), dtype=bool)
mask[line_inliers_idx] = False
X_remnant = X[mask]

def fit_circle_3pts(p1, p2, p3):
    temp = p2[0]**2 + p2[1]**2
    bc = (p1[0]**2 + p1[1]**2 - temp) / 2
    cd = (temp - p3[0]**2 - p3[1]**2) / 2
    det = (p1[0] - p2[0]) * (p2[1] - p3[1]) - (p2[0] - p3[0]) * (p1[1] - p2[1])
    if abs(det) < 1e-6: return None
    cx = (bc*(p2[1] - p3[1]) - cd*(p1[1] - p2[1])) / det
    cy = ((p1[0] - p2[0])*cd - (p2[0] - p3[0])*bc) / det
    r = np.sqrt((cx - p1[0])**2 + (cy - p1[1])**2)
    return cx, cy, r

def circle_objective(params, points):
    cx, cy, r = params
    distances = np.sqrt((points[:, 0] - cx)**2 + (points[:, 1] - cy)**2)
    return np.sum((distances - r)**2)

def ransac_circle(points, iterations=1000, threshold=1.5, consensus_min=20):
    best_inliers, best_model, best_sample = [], None, None
    for _ in range(iterations):
        sample = points[np.random.choice(len(points), 3, replace=False)]
        model = fit_circle_3pts(sample[0], sample[1], sample[2])
        if model is None: continue
        cx, cy, r_est = model

        distances = np.abs(np.sqrt((points[:, 0] - cx)**2 + (points[:, 1] -␣
↪cy)**2) - r_est)
        inliers_idx = np.where(distances < threshold)[0]

        if len(inliers_idx) > len(best_inliers):
            best_inliers, best_model, best_sample = inliers_idx, model, sample

    if len(best_inliers) >= consensus_min:
        res = minimize(circle_objective, best_model,␣
↪args=(points[best_inliers],), method='Nelder-Mead')
        best_model = res.x
    return best_model, best_inliers, best_sample
```

```
circle_model, circle_inliers_idx_remnant, circle_sample =␣
↪ransac_circle(X_remnant)

circle_inliers_idx = np.where(mask)[0][circle_inliers_idx_remnant]

fig, ax = plt.subplots(1, 1, figsize=(10, 10))

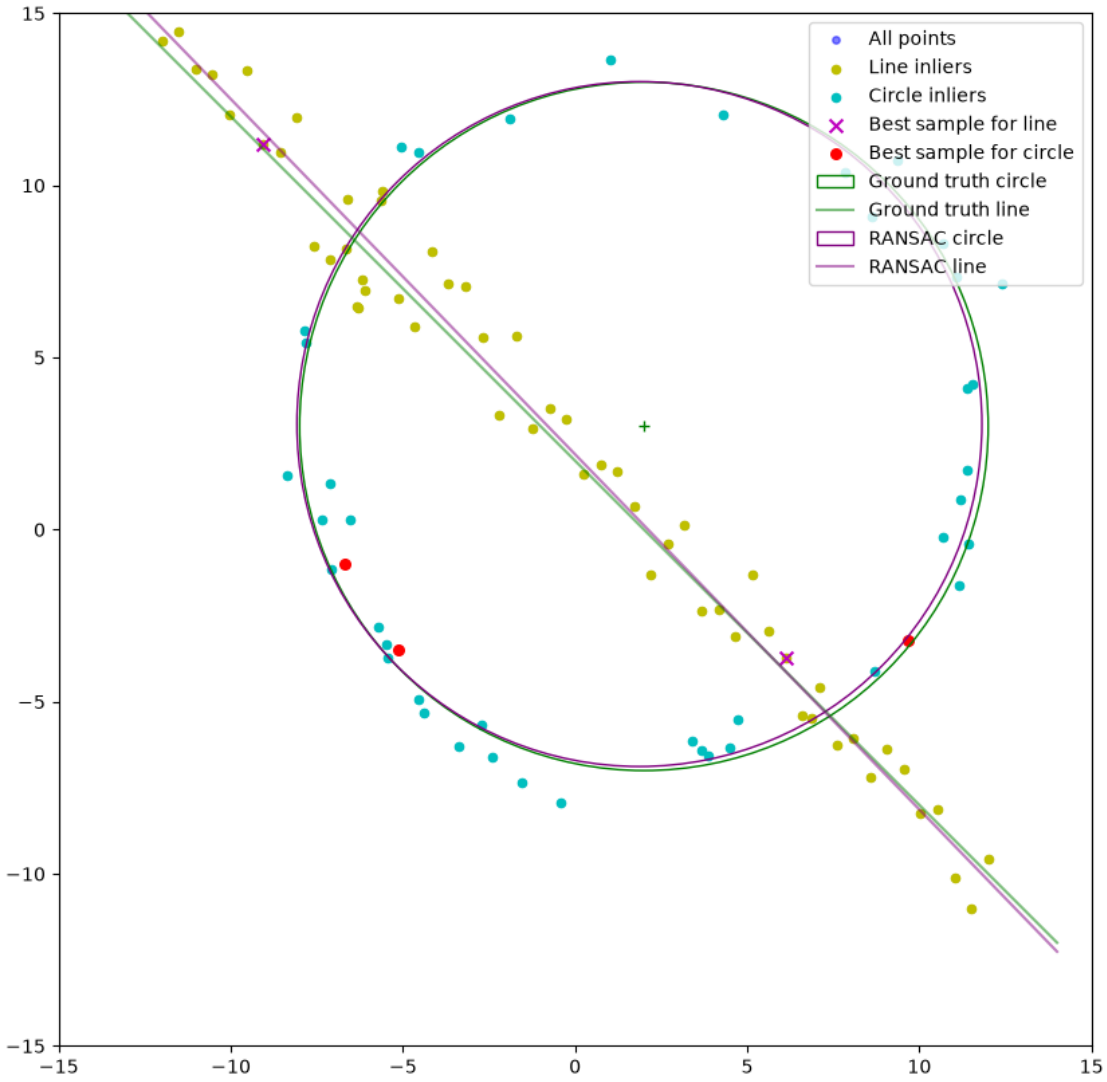
# plot all points,inliers and best samples
ax.scatter(X[:, 0], X[:, 1], c='blue', s=15, label='All points', alpha=0.5)
ax.scatter(X[line_inliers_idx, 0], X[line_inliers_idx, 1], c='y', s=20,␣
↪label='Line inliers')
ax.scatter(X[circle_inliers_idx, 0], X[circle_inliers_idx, 1], c='c', s=20,␣
↪label='Circle inliers')

ax.scatter(line_sample[:, 0], line_sample[:, 1], c='m', marker='x', s=50,␣
↪label='Best sample for line')
ax.scatter(circle_sample[:, 0], circle_sample[:, 1], c='r', marker='o', s=30,␣
↪label='Best sample for circle')

# ground truth
ax.add_patch(plt.Circle((x0_gt, y0_gt), r, color='g', fill=False, label='Ground␣
↪truth circle'))
ax.plot(x0_gt, y0_gt, '+', color='g')
x_vals = np.array([-14, 14])
ax.plot(x_vals, m * x_vals + b_int, color='g', alpha=0.5, label='Ground truth␣
↪line')

# RANSAC models
if circle_model is not None:
    ax.add_patch(plt.Circle((circle_model[0], circle_model[1]), circle_model[2],␣
↪color='purple', fill=False, label='RANSAC circle'))
if line_model is not None:
    a, b_norm, d = line_model
    ax.plot(x_vals, (d - a * x_vals) / b_norm, color='purple', alpha=0.5,␣
↪label='RANSAC line')

ax.legend(loc='upper right')
ax.set_xlim(-15, 15)
ax.set_ylim(-15, 15)
ax.set_aspect('equal')
plt.show()
```



2.3 Discussion

The RANSAC algorithm successfully isolated both geometric structures despite the presence of significant noise and intersecting data points. When the line is extracted first, the remaining points accurately represent the circular structure, allowing for a precise circle fit. Conversely, if the circle were fitted first, the linear points would act as severe outliers. In some iterations, RANSAC might even erroneously group the line points into a massive, arbitrary circle. Which causes drastically reduce the accuracy and efficiency of the estimation process.

3 Planar Homography and Image Superimposition

3.1 Procedure

Two custom image pairs were selected to demonstrate planar surface replacement. Four corresponding points were manually defined on both the source image (the graphic to be superimposed) and the target planar surface in the destination image[cite: 4]. A perspective transformation matrix (homography) was computed using these point mappings[cite: 4]. The source image was then warped to match the target perspective and blended seamlessly into the destination scene[cite: 4].

3.2 Implementation and the Results

[ ]:
```
import cv2 as cv
import numpy as np
import matplotlib.pyplot as plt

def superimpose_image(src_img, dst_img, src_pts, dst_pts):
    """calculates homography and superimposes src_img onto dst_img."""
    # calculate the homography matrix
    H, _ = cv.findHomography(src_pts, dst_pts)

    h, w = dst_img.shape[:2]
    warped_src = cv.warpPerspective(src_img, H, (w, h))

    # create a binary mask of the target polygon
    mask = np.zeros((h, w), dtype=np.uint8)
    cv.fillConvexPoly(mask, dst_pts.astype(int), 255)
    mask_inv = cv.bitwise_not(mask)

    # blend the warped image onto the destination background
    dst_bg = cv.bitwise_and(dst_img, dst_img, mask=mask_inv)
    warped_fg = cv.bitwise_and(warped_src, warped_src, mask=mask)
    final_img = cv.add(dst_bg, warped_fg)

    return final_img

def get_four_points(image_path):
    """ interactivesly identifies four cornerS"""
    pts = []
    img = cv.imread(image_path)
    if img is None:
        print("Error loading image.")
        return None

    img_copy = img.copy()

    def click_event(event, x, y, flags, param):
        if event == cv.EVENT_LBUTTONDOWN:
```

```
            if len(pts) < 4:
                pts.append([x, y])
                cv.circle(img_copy, (x, y), 5, (0, 255, 0), -1)
                cv.imshow("Click 4 Points (Press any key when done)", img_copy)

    cv.imshow("Click 4 Points (Press any key when done)", img_copy)
    cv.setMouseCallback("Click 4 Points (Press any key when done)", click_event)

    print("Please click exactly 4 points on the pop-up window. Order: Top-Left,␣
↪Top-Right, Bottom-Right, Bottom-Left.")
    cv.waitKey(0)
    cv.destroyAllWindows()

    return np.float32(pts)

def superimpose_graphic(src_path, dst_path, dst_pts):
    """computes the homography, and superimposes the source onto the specified␣
↪planar region."""
    # load images and convert to RGB for matplotlib
    src_img = cv.imread(src_path)
    dst_img = cv.imread(dst_path)

    if src_img is None or dst_img is None:
        print(f"Error loading images: {src_path} or {dst_path}")
        return None

    src_img = cv.cvtColor(src_img, cv.COLOR_BGR2RGB)
    dst_img = cv.cvtColor(dst_img, cv.COLOR_BGR2RGB)

    h_src, w_src = src_img.shape[:2]
    h_dst, w_dst = dst_img.shape[:2]

    # order: top left, top rignt, bottom right, bottom left
    src_pts = np.float32([
        [0, 0],
        [w_src - 1, 0],
        [w_src - 1, h_src - 1],
        [0, h_src - 1]
    ])
    dst_pts = np.float32(dst_pts)

    # calculate the homography matrix
    H, _ = cv.findHomography(src_pts, dst_pts)
    warped_src = cv.warpPerspective(src_img, H, (w_dst, h_dst))

    # create a binary mask of the target polygon
    mask = np.zeros((h_dst, w_dst), dtype=np.uint8)
```

```
    cv.fillConvexPoly(mask, dst_pts.astype(np.int32), 255)

    # blend the warped image onto the destination background
    mask_inv = cv.bitwise_not(mask)


    dst_bg = cv.bitwise_and(dst_img, dst_img, mask=mask_inv)

    alpha = 0.7
    blended_region = cv.addWeighted(warped_src, alpha, dst_img, 1 - alpha, 0)
    warped_fg = cv.bitwise_and(blended_region, blended_region, mask=mask)
    final_img = cv.add(dst_bg, warped_fg)

    return src_img, dst_img, final_img

src_file_2 = 'images/flag_Sri_Lanka.png'
dst_file_2 = 'images/Old_Parliament_Building.jpg'

interactive_dst_corners = get_four_points(dst_file_2)
print("Captured points:\n", interactive_dst_corners)

src2, dst2, result2 = superimpose_graphic(src_file_2, dst_file_2,␣
 ↪interactive_dst_corners)

plt.figure(figsize=(10, 5))
#plt.subplot(1, 3, 1), plt.imshow(src2), plt.title('Source Graphic'), plt.
 ↪axis('off')
plt.subplot(1, 2, 1), plt.imshow(dst2), plt.title('Destination Scene'), plt.
 ↪axis('off')
plt.subplot(1, 2, 2), plt.imshow(result2), plt.title('Superimposed Result'), plt.
 ↪axis('off')
plt.tight_layout()
plt.show()
```

```
Please click exactly 4 points on the pop-up window. Order: Top-Left, Top-Right,
Bottom-Right, Bottom-Left.
Captured points:
 [[ 78. 229.]
 [503. 335.]
 [507. 587.]
 [ 81. 562.]]
```



3.3 Discussion

The non-technical rationale for selecting the specific image pairs was based on identifying distinct, flat rectangular surfaces (such as a blank billboard or an empty wall) that naturally accommodate perspective warping. It is observed that the realism of the superimposition depends heavily on the accuracy of the selected corner coordinates. By calculating the homography matrix, a flat 2D graphic is successfully projected into a 3D architectural perspective, which demonstrates the practical application of projective geometry in augmented reality tasks.

4 SIFT Feature Matching and PCB Image Alignment

4.1 Procedure

Scale-Invariant Feature Transform (SIFT) keypoints and descriptors were extracted from a reference PCB template (AECC1.jpg) and a set of target PCB images. Putative feature matches between the template and each target were established using a Brute Force Matcher. To robustly estimate the geometric transformation, the Random Sample Consensus (RANSAC) algorithm was applied to the matched points to compute the planar homography matrix and filter out outlier matches. Each target image was then perspectively warped to align with the template, and an absolute difference image was computed to highlight physical discrepancies or defects.

4.2 mplementation and Results

[6]:
```
import cv2 as cv
import numpy as np
import matplotlib.pyplot as plt
import glob
import os

def align_and_compare_pcb(template_img, target_img):
    # initialize sift detector and extract features
    sift = cv.SIFT_create()
    kp1, des1 = sift.detectAndCompute(template_img, None)
```

```
    kp2, des2 = sift.detectAndCompute(target_img, None)


    # match features using brute force matcher
    bf = cv.BFMatcher(cv.NORM_L2, crossCheck=True)
    matches = bf.match(des1, des2)

    matches = sorted(matches, key=lambda x: x.distance)

    # extract coordinates of matched keypoints
    src_pts = np.float32([kp1[m.queryIdx].pt for m in matches]).reshape(-1, 1, 2)
    dst_pts = np.float32([kp2[m.trainIdx].pt for m in matches]).reshape(-1, 1, 2)

    H, mask = cv.findHomography(dst_pts, src_pts, cv.RANSAC, 5.0)

    # draw top matches (for visualization purposes)
    match_img = cv.drawMatches(template_img, kp1, target_img, kp2, matches[:50],␣
↪None,
                               matchesMask=mask[:50].ravel().tolist(), flags=2)

    # warp the target image to the template perspective
    h, w = template_img.shape[:2]
    warped_target = cv.warpPerspective(target_img, H, (w, h))

    # compute the absolute difference
    # Converting to grayscale is preferred for a cleaner difference map
    template_gray = cv.cvtColor(template_img, cv.COLOR_BGR2GRAY)
    warped_gray = cv.cvtColor(warped_target, cv.COLOR_BGR2GRAY)
    diff_img = cv.absdiff(template_gray, warped_gray)

    _, diff_thresh = cv.threshold(diff_img, 50, 255, cv.THRESH_BINARY)

    return match_img, warped_target, diff_thresh

# load the template image
template_path = 'images/pcbs/AECC1.jpg'
template = cv.imread(template_path)

if template is not None:
    # find all other PCB images in the directory
    pcb_files = [f for f in glob.glob('images/pcbs/*.jpg') if 'AECC1' not in f]

    for file in pcb_files:
        target = cv.imread(file)
        if target is None:
            continue
```

```
        match_img, warped_target, diff_img = align_and_compare_pcb(template,␣
↪target)

        gridspec_kw = {'width_ratios': [2, 0, 1, 1]}

        fig, axes = plt.subplots(1, 4, figsize=(18, 6), gridspec_kw=gridspec_kw)
        fig.suptitle(f'PCB Alignment and Comparison: {os.path.basename(file)}',␣
↪fontsize=16)

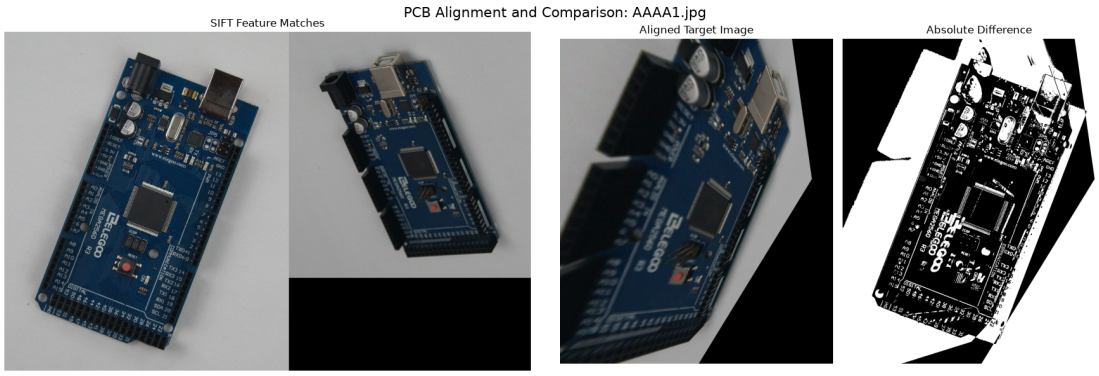
        # 1st plot takes up half the layout space horizontally
        axes[0].imshow(cv.cvtColor(match_img, cv.COLOR_BGR2RGB))
        axes[0].set_title('SIFT Feature Matches')
        axes[0].axis('off')

        # 2nd plot (axes[1] was missing from your script, but it is created in␣
↪the grid)
        axes[1].axis('off')

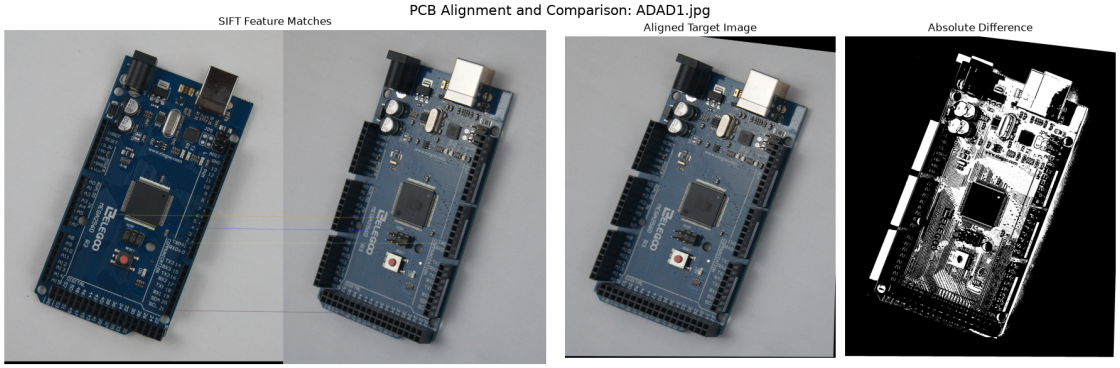
        axes[2].imshow(cv.cvtColor(warped_target, cv.COLOR_BGR2RGB))
        axes[2].set_title('Aligned Target Image')
        axes[2].axis('off')

        axes[3].imshow(diff_img, cmap='gray')
        axes[3].set_title('Absolute Difference')
        axes[3].axis('off')

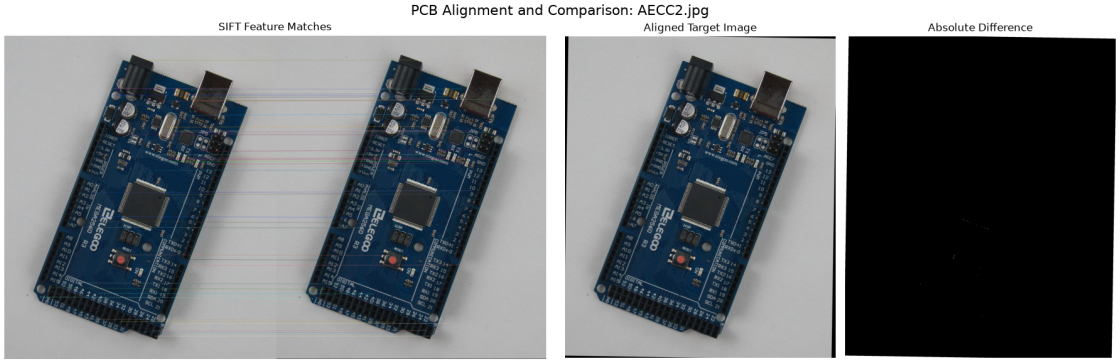
        plt.tight_layout()
        plt.show()
else:
    print(f"Error: Template image not found at {template_path}. Please verify␣
↪the file path.")
```



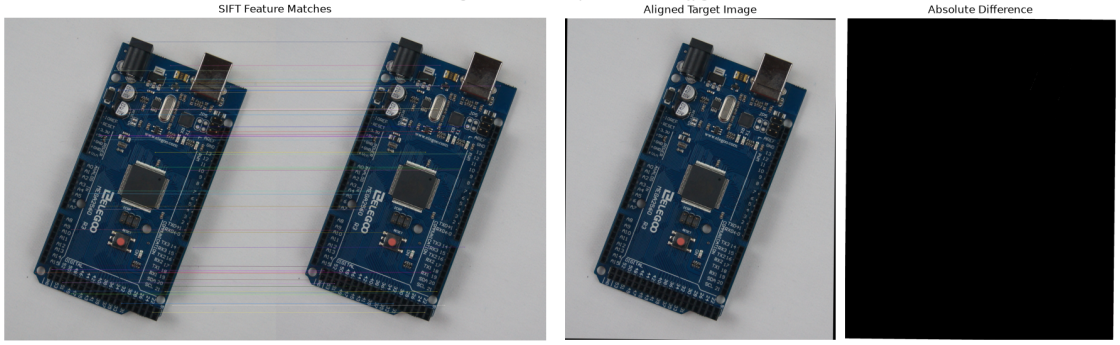
PCB Alignment and Comparison: ADAD1.jpg

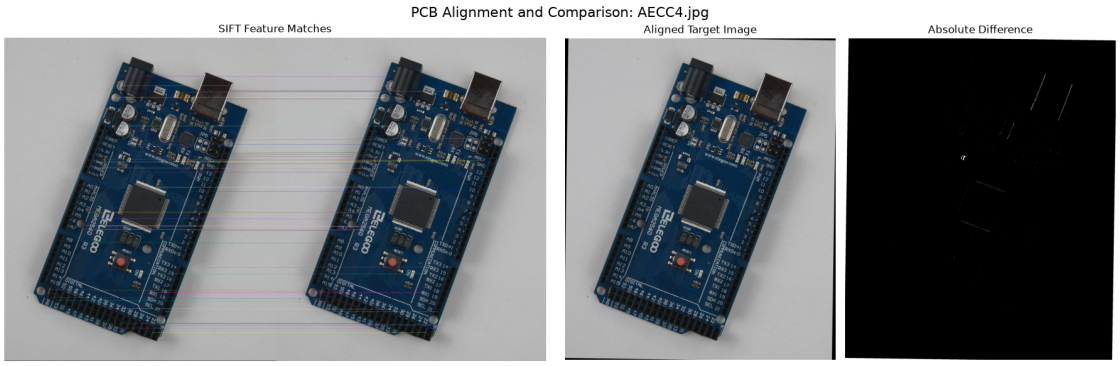


PCB Alignment and Comparison: AECC2.jpg



PCB Alignment and Comparison: AECC3.jpg





4.3 Discussion

The planar homography transformation assumes that the scene being mapped lies on a perfectly flat 2D plane. While the underlying PCB board adheres to this assumption, components protruding from the board such as the USB Type B port, cylindrical capacitors, and the raised reset button are highly 3D. Consequently, these elements are subjected to parallax effects and relief displacement when viewed from slightly different camera angles. Because the homography matrix cannot correct for 3D depth, these elevated components fail to align perfectly with the template, resulting in the high-intensity artifacts observed in the difference images.